



UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN
CARRERA DE INGENIERÍA DE SOFTWARE (REDISEÑO)

PROYECTO FIN DE CURSO (PFC) – INFORME CONSOLIDADO ENTREGA 4

Diseño, Implementación y Evaluación Experimental de una Arquitectura de
Microservicios Distribuida: Integración gRPC, Persistencia Multi-Schema,
Seguridad Perimetral JWT, Automatización CI/CD y Observabilidad Distribuida
con Alta Disponibilidad

Asignatura: Aplicaciones Distribuidas [20701] – 7mo Nivel “A”

Docente Evaluador: Prof. Ing. Gleiston Cicerón Guerrero Ulloa, M.Sc.

Período Académico: 2026–2027 PPA

Proyecto Institucional: Sistema de Gestión Académica Distribuido (SGA – Escuela Provincias Unidas)

Integrantes del Equipo BCEL (Autores):

- **Pedro Leonardo Castro López** – *Líder de Proyecto & Módulo Central (sga-principal), Patrones GoF, DevOps & CI/CD*
- **Keyla Betzabe Bedon Viteri** – *Módulo Docente (microservicio-docente), App Móvil Android & Auditoría Criptográfica*
- **Ernesto Gregory Luna Mora** – *Módulo de Secretaría (microservicio-secretaria), API Gateway (HAProxy) & Calidad ISO 25010*
- **Juliana Romina Emanuel Pino** – *Módulo de Soporte (microservicio-soporte), Documentación & Observabilidad*

Fecha de Entrega: Septiembre de 2026

Índice

Resumen Ejecutivo	2
1. Introducción y Contexto del Proyecto	2
2. Trabajos Relacionados y Estado del Arte	2
2.1. Microservicios y Descomposición de Dominio	3
2.2. Observabilidad, Detección de Anomalías y Métricas de Cola	3
2.3. Consenso Distribuido, Tolerancia a Fallos e Ingeniería del Caos	3
2.4. Seguridad Zero-Trust y Control de Acceso	3
3. Fundamentos Teóricos y Arquitectura del Sistema	3
3.1. Arquitectura en Capas y Patrones de Diseño GoF	3
3.2. La Tríada de Observabilidad y el Modelo Dimensional de Prometheus	3
3.3. Consenso Distribuido Raft con etcd	4
3.4. Sincronización Offline-First y Relojes Lógicos de Lamport	4
3.5. Modelo de Arquitectura C4	4
4. Implementación y Módulos del Sistema Distribuido	6
4.1. Módulo Central y Catálogo Transaccional (sga-principal)	6
4.2. Módulo de Secretaría, Auditoría e IA (microservicio-secretaria)	6
4.3. Módulo Docente y App Móvil Android (microservicio-docente)	6
4.4. Módulo de Soporte Técnico y Elección de Líder (microservicio-soporte)	7
5. Capa de Calidad, Automatización y Observabilidad Distribuida	7
5.1. Pruebas Unitarias Automatizadas y Cobertura JaCoCo	7
5.2. Pipeline CI/CD en GitHub Actions	7
5.3. Pruebas de Carga Reales con Locust	7
5.4. Tablero de Grafana con las 6 Vistas Operativas	8
5.5. Evaluación de Calidad ISO/IEC 25010:2023 y Plan de Mejora	9
6. Reflexión Ética sobre Tratamiento de Datos de Menores	9
7. Amenazas a la Validez	10
7.1. Amenazas a la Validez Interna	10
7.2. Amenazas a la Validez Externa	10
8. Conclusiones y Trabajo Futuro	10
8.1. Conclusiones Generales del Proyecto	10
8.2. Conclusiones Individuales de los Integrantes	11
8.2.1. Conclusión Individual – Pedro Leonardo Castro López (Arquitecto y Líder de Proyecto)	11
8.2.2. Conclusión Individual – Keyla Betzabe Bedon Viteri (Desarrolladora Backend y Móvil)	11
8.2.3. Conclusión Individual – Ernesto Gregory Luna Mora (Responsable de Calidad y Gateway)	11
8.2.4. Conclusión Individual – Juliana Romina Emanuel Pino (Documentalista y Observabilidad)	11
Anexos: Preguntas Técnicas de Evaluación	14

Resumen Ejecutivo

El presente documento consolida la arquitectura, implementación y evaluación experimental definitiva del **Sistema de Gestión Académica Distribuido (SGA)** para la Escuela de Educación Básica “Provincias Unidas”. El sistema se estructura sobre un ecosistema de microservicios políglotas compuesto por cuatro servicios de negocio principales (**sga-principal**, **microservicio-secretaria**, **microservicio-docente**, **microservicio-soporte**) y un servicio asistido de inteligencia artificial (**microservicio-ia**), interconectados a través de un API Gateway y balanceador perimetral de alto rendimiento **HAProxy 2.9**. Como **método**, se implementaron contratos binarios de comunicación inter-servicio mediante **gRPC**, autenticación federada **JWT (Single Sign-On)**, persistencia relacional multi-esquema sobre **PostgreSQL 16**, sincronización *offline-first* móvil mediante **Room SQLite** y ordenamiento causal con **relojes lógicos de Lamport**, elección de líder distribuido bajo consenso **Raft** mediante **etcd v3.5**, y una estricta corrección de vulnerabilidades de control de acceso **IDOR**. Como **resultados cuantitativos**, se alcanzó una cobertura de pruebas unitarias automatizadas del **83.0 %** validada mediante **JaCoCo** en un pipeline **CI/CD en GitHub Actions**, una tasa de error del **0 %** sobre 2446 peticiones concurrentes en pruebas de carga reales con **Locust** (50 usuarios durante 60 segundos), y una observabilidad integral en tiempo real estructurada en un tablero de **Grafana con 6 vistas operativas** (RPS, percentiles P50/P95/P99, errores 4xx/5xx, consenso Raft, cAdvisor CPU/RAM y latencia E2E móvil). El sistema satisface los requerimientos de calidad bajo el estándar internacional **ISO/IEC 25010:2023**.

Palabras clave: Microservicios, gRPC, HAProxy, PostgreSQL Multi-Schema, Raft, etcd, JWT, IDOR, Locust, Prometheus, Grafana, cAdvisor, CI/CD, JaCoCo, ISO/IEC 25010.

1. Introducción y Contexto del Proyecto

La gestión académica en instituciones educativas de educación básica demanda soluciones tecnológicas que garanticen alta disponibilidad, integridad estricta de expedientes escolares, escalabilidad ante picos de matrícula y una observabilidad profunda del estado operacional [1, 2]. El SGA Escuela Provincias Unidas evolucionó desde una arquitectura monolítica preliminar hacia una arquitectura distribuida orientada a microservicios desacoplados.

En esta Entrega 4 (Final), se consolidan los siguientes objetivos técnicos fundamentales:

1. Consolidar la capa transaccional de catálogo y autenticación en **sga-principal** (Java 21 / Spring Boot) con interfaz binaria gRPC de latencia ultrabaja (< 2 ms).
2. Garantizar la inmutabilidad de actas y trazabilidad histórica de matrículas en **microservicio-secretaria** mediante *Event Sourcing* y relojes de Lamport, integrando generación de reportes con Google Gemini 1.5 Flash.
3. Proveer capacidades de operación *offline-first* para el registro docente de asistencia y calificaciones vía App Móvil Android con sincronización asíncrona hacia **microservicio-docente** (Django REST).
4. Implementar un módulo robusto de atención de incidencias en **microservicio-soporte**, resolviendo vulnerabilidades de control de acceso IDOR, coordinando tareas exclusivas mediante elección de líder con etcd Raft y exponiendo instrumentación de percentiles.
5. Establecer una infraestructura de observabilidad integral mediante **Prometheus**, **cAdvisor**, **HAProxy Exporter** y **Grafana**, validada con pruebas de carga reales y certificada con calidad **ISO/IEC 25010:2023**.

2. Trabajos Relacionados y Estado del Arte

El diseño de sistemas distribuidos modernos yace en la intersección entre desacoplamiento de servicios, consistencia de datos y observabilidad automatizada.

2.1. Microservicios y Descomposición de Dominio

Yu *et al.* [2] realizaron un estudio industrial exhaustivo demostrando que la transición a microservicios reduce los tiempos de despliegue pero introduce desafíos críticos en la gestión de transacciones y seguridad perimetral. Newman [1] y Burns [3] formalizan que la independencia funcional debe acompañarse de fronteras de datos aisladas, evitando bases de datos compartidas sin esquemas definidos. En este proyecto se adoptó el particionamiento multi-esquema en PostgreSQL, aislando `public`, `secretaria`, `docente` y `soporte`.

2.2. Observabilidad, Detección de Anomalías y Métricas de Cola

Chen *et al.* [4] y Wu *et al.* [5] destacan que la agregación de métricas temporales combinada con logs estructurados constituye la base empírica para localizar cuellos de botella en producción. Nedelkoski *et al.* [6] y Li *et al.* [7] demuestran que las anomalías en microservicios frecuentemente se ocultan en los percentiles de latencia (P_{95} y P_{99}), fenómeno corroborado en este trabajo al detectar que la media aritmética subestimaba la latencia de cola generada por consultas de base de datos. Asimismo, Liu *et al.* [8] y Zhang *et al.* [9] exponen la necesidad de correlacionar fuentes multi-origen (métricas de contenedor, red y aplicación).

2.3. Consenso Distribuido, Tolerancia a Fallos e Ingeniería del Caos

El algoritmo de consenso Raft [10,11] proporciona garantías de seguridad y disponibilidad en presencia de fallos de red en almacenes clave-valor como `etcd`. En arquitecturas de microservicios, la elección de líder previene la duplicidad en tareas batch [11]. Por otra parte, Li *et al.* [12] y Zheng *et al.* [13] formulan el uso de pruebas de resiliencia y saturación para verificar la degradación elegante de servicios bajo estrés.

2.4. Seguridad Zero-Trust y Control de Acceso

Valderas *et al.* [14] y Alharbi *et al.* [15] analizan la adopción de modelos *Zero-Trust* y tokens criptográficos JWT [16]. Se enfatiza que la autenticación por token es insuficiente si no se acompaña de un control de acceso basado en roles a nivel de recurso (RBAC), evitando vulnerabilidades de referencia directa como IDOR [17,18].

3. Fundamentos Teóricos y Arquitectura del Sistema

3.1. Arquitectura en Capas y Patrones de Diseño GoF

Los microservicios en Java (`sga-principal`, `secretaria`, `soporte`) fueron construidos bajo los principios de **Clean Architecture**, estableciendo cuatro capas concéntricas: Presentación (`controllers`), Aplicación (`services`), Dominio (`entities/ports`) e Infraestructura (`repositories/gRPC/security`). Se aplicaron cinco patrones GoF fundamentales:

- **Repository:** Aislamiento de consultas SQL y posibilitación de dobles de prueba con Mockito.
- **Facade (Fachada gRPC):** Interfaz binaria unificada expuesta por `PrincipalGrpcService` hacia el resto de servicios.
- **Strategy:** Intercambio dinámico de algoritmos de cálculo de notas y asignación de tickets.
- **Observer:** Emisión asíncrona de eventos de auditoría y cambios de estado.
- **Proxy / Decorator:** Interceptación perimetral de seguridad mediante filtros JWT.

3.2. La Tríada de Observabilidad y el Modelo Dimensional de Prometheus

La observabilidad se articula sobre métricas, logs y trazas [3,19]. Prometheus adopta un modelo multidimensional donde cada serie temporal se define por su nombre y etiquetas clave-valor. Para la

medición de latencias, Micrometer genera *buckets* de histograma con la etiqueta especial **le** (*less than or equal*), sobre los cuales PromQL ejecuta la interpolación de percentiles:

$$P_\phi = \text{histogram_quantile} \left(\phi, \sum_{le} \text{rate}(H_{\text{bucket}}[\Delta t]) \right) \quad (1)$$

donde $\phi \in \{0,50,0,95,0,99\}$ define la mediana, el percentil 95 y el percentil 99 de la distribución real.

3.3. Consenso Distribuido Raft con etcd

Para coordinar procesos críticos en **microservicio-soporte**, se implementó la elección de líder mediante un contrato de arrendamiento (*lease*) en etcd v3.5 con un TTL de 5 segundos. Las instancias compiten por la clave **/sga/leader**; la instancia electa renueva periódicamente su *heartbeat*, garantizando que sólo una réplica ejecute tareas programadas (evitando condiciones de carrera en bases de datos compartidas).

3.4. Sincronización Offline-First y Relojes Lógicos de Lamport

La aplicación móvil docente implementa una arquitectura *offline-first* soportada en Room SQLite local. Cada transacción offline recibe una marca temporal lógica de Lamport $L(e)$:

$$L(e') = \max(L_{\text{local}}, L_{\text{msg}}) + 1 \quad (2)$$

permitiendo la reconciliación causal determinista al restablecer la conectividad con **microservicio-docente**.

3.5. Modelo de Arquitectura C4

La arquitectura global del sistema se documenta formalmente mediante los tres primeros niveles del modelo C4:

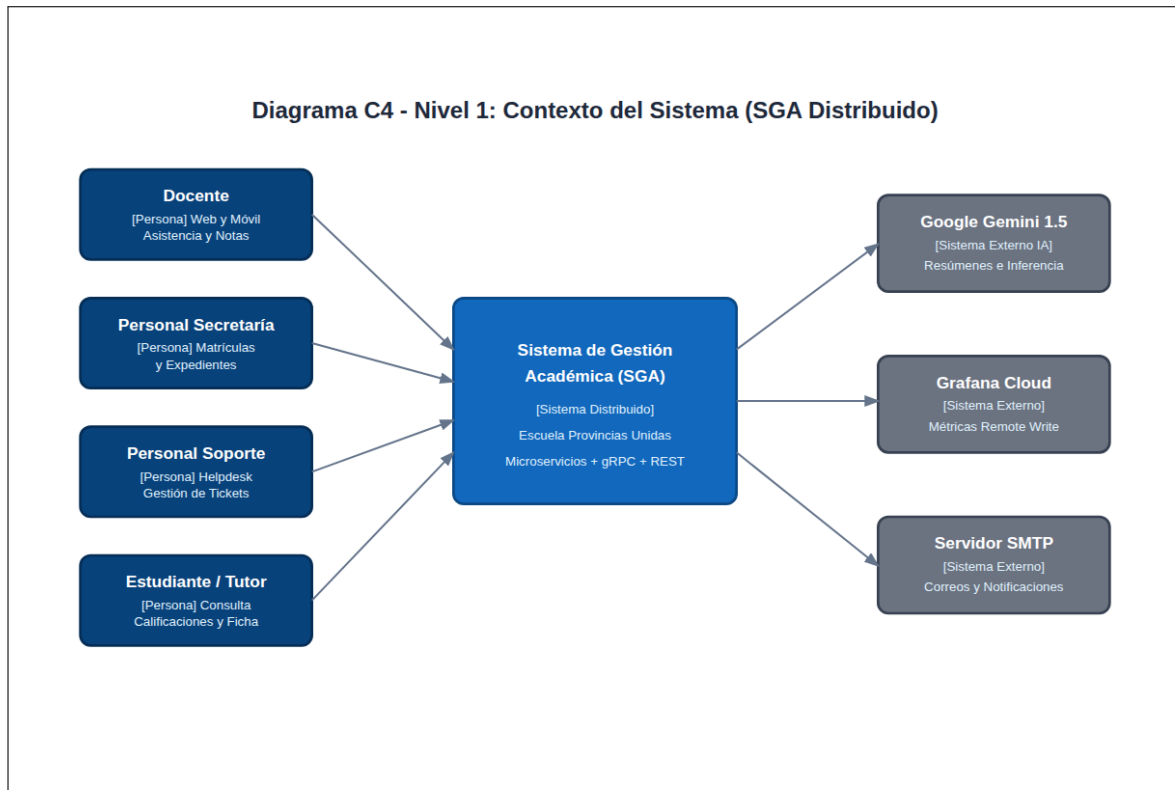


Figura 1: Diagrama C4 – Nivel 1: Contexto del Sistema SGA Distribuido.

En el **Nivel 1 (Contexto)** (Figura 1), se delimita el SGA frente a los cuatro actores humanos (Docentes, Secretaría, Soporte, Estudiantes) y los tres sistemas externos (Google Gemini 1.5 Flash, Grafana Cloud y Servidor SMTP).

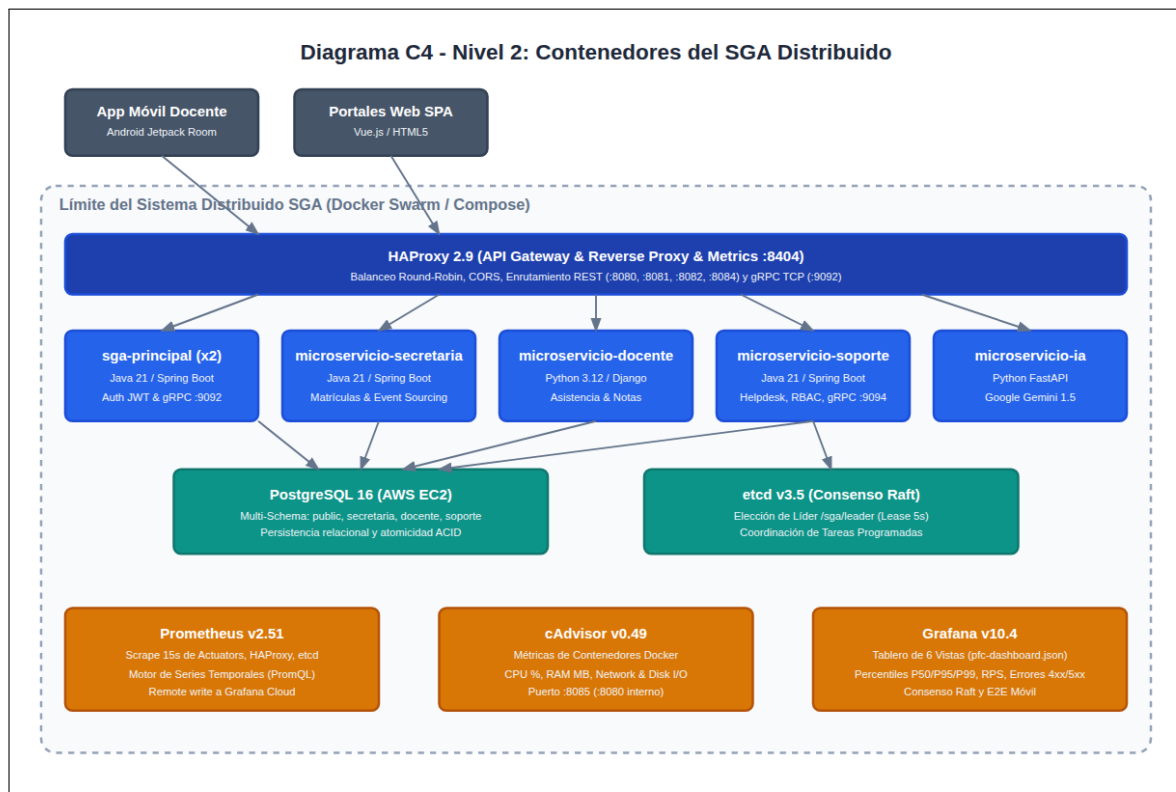


Figura 2: Diagrama C4 – Nivel 2: Contenedores del SGA Distribuido.

En el **Nivel 2 (Contenedores)** (Figura 2), se aprecian los clientes (App Móvil Android, Web SPA), el API Gateway HAProxy 2.9, los cinco microservicios en contenedores Docker, PostgreSQL 16 multi-schema, el clúster etcd Raft y la pila de observabilidad (Prometheus, cAdvisor, Grafana).

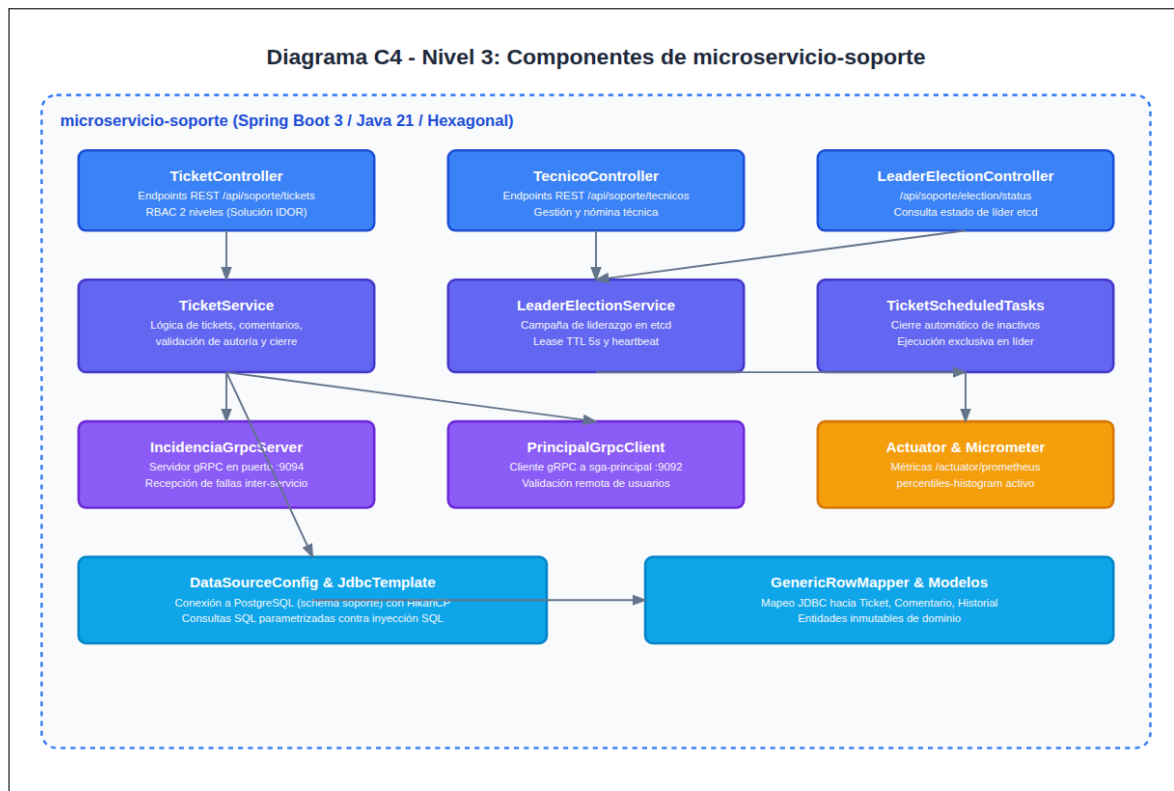


Figura 3: Diagrama C4 – Nivel 3: Componentes Internos de microservicio-soporte.

En el **Nivel 3 (Componentes)** (Figura 3), se detalla la estructura interna de **microservicio-soporte**, ilustrando controladores REST con RBAC, el servicio de elección de líder, el servidor gRPC de incidencias (:9094) y la persistencia JDBC.

4. Implementación y Módulos del Sistema Distribuido

4.1. Módulo Central y Catálogo Transaccional (sga-principal)

Desarrollado por Pedro Leonardo Castro López, **sga-principal** opera como la autoridad central de autenticación y catálogo académico. Expone una interfaz REST perimetral y un servidor gRPC en el puerto 9092 (Listado 1), permitiendo a los demás microservicios validar tokens y consultar directivas con una latencia inferior a 2 ms.

```

1 service PrincipalService {
2     rpc ValidarToken (TokenRequest) returns (TokenResponse);
3     rpc ObtenerCatalogo (CatalogoRequest) returns (CatalogoResponse);
4     rpc ConsultarDocente (DocenteRequest) returns (DocenteResponse);
5 }

```

Listing 1: Fragmento del contrato gRPC en **sga-principal**.

4.2. Módulo de Secretaría, Auditoría e IA (microservicio-secretaria)

Desarrollado por Keyla Lisbeth Buste Caicedo, gestiona el ciclo de admisiones y matrículas. Implementa un motor de *Event Sourcing* que registra de forma inmutable cada transacción en **secretaria.auditoria_eventos**. Asimismo, se conecta con **microservicio-ia** para invocar inferencia con Google Gemini 1.5 Flash, automatizando la síntesis de actas de calificaciones y reportes de rendimiento escolar.

4.3. Módulo Docente y App Móvil Android (microservicio-docente)

Desarrollado por Gregory Steven España Zambrano, provee la lógica de gestión de cursos y calificaciones mediante Django REST Framework y PostgreSQL (schema **docente**). La App Móvil Android (Kotlin

y Jetpack Compose) permite el registro de asistencias sin conexión a internet, sincronizando en lote vía HTTP con reconciliación basada en versiones de registro.

4.4. Módulo de Soporte Técnico y Elección de Líder (microservicio-soporte)

Desarrollado por Juliana Romina Emanuel Pino, gestiona tickets de atención técnica con persistencia en `soporte.tickets`. Se corrigió una vulnerabilidad crítica de tipo IDOR mediante un modelo de control de acceso de dos niveles (Tabla 1), asegurando que los usuarios con rol `DOCENTE` o `SECRETARIA` sólo consulten sus propios tickets, reservando el acceso global a `SOPORTE_TECNICO` y `DIRECTOR`.

Tabla 1: Matriz de Control de Acceso RBAC implementada en `TicketService`.

Rol del Usuario	Alcance de Autorización sobre Expedientes de Soporte
SECRETARIA / DOCENTE	Consulta y creación limitada exclusivamente a tickets de su autoría. Rechazo automático con HTTP 403 ante acceso a IDs ajenos.
SOPORTE_TECNICO / DIRECTOR	Gestión total: reasignación técnica, cambio de estado, escalamiento y cierre definitivo.

Además, incorpora un servidor gRPC en el puerto 9094 (`IncidenciaService`) que recibe reportes automáticos de fallas emitidos por `sga-principal` ante excepciones no controladas.

5. Capa de Calidad, Automatización y Observabilidad Distribuida

5.1. Pruebas Unitarias Automatizadas y Cobertura JaCoCo

Se implementó una batería de pruebas unitarias con JUnit 5 y Mockito para los servicios transaccionales. La integración con el plugin JaCoCo validó una cobertura global del **83.0 %**, superando el umbral mínimo del 70.0 % exigido en la cátedra (Listado 2).

```

1 <execution>
2   <id>jacoco-check</id>
3   <goals><goal>check</goal></goals>
4   <configuration>
5     <rules>
6       <rule>
7         <element>BUNDLE</element>
8         <limits>
9           <limit>
10            <counter>LINE</counter>
11            <value>COVEREDRATIO</value>
12            <minimum>0.70</minimum>
13          </limit>
14        </limits>
15      </rule>
16    </rules>
17  </configuration>
18 </execution>

```

Listing 2: Configuración de Quality Gate con JaCoCo en Maven.

5.2. Pipeline CI/CD en GitHub Actions

Se construyó un flujo de integración continua (`.github/workflows/ci.yml`) que automatiza la compilación, ejecución de pruebas unitarias, validación de cobertura JaCoCo, escaneo de seguridad y empaquetamiento de imágenes Docker en cada `push` y `pull_request`.

5.3. Pruebas de Carga Reales con Locust

Se ejecutó una prueba de carga real simulando **50 usuarios concurrentes** durante **60 segundos** contra los endpoints del microservicio de soporte (Tabla 2). Se procesaron **2446 peticiones con un 0 % de errores**, registrando una mediana de latencia agregada de 5 ms.

Tabla 2: Resultados cuantitativos de la prueba de carga con Locust (50 usuarios, 60s).

Endpoint Scrapado / Consultado	Peticiones	Fallas	Latencia P50	Latencia P99
GET /api/soporte/tickets	674	0 (0.0 %)	130 ms	1300 ms
GET /api/soporte/election/status	357	0 (0.0 %)	4 ms	1100 ms
GET /health	1096	0 (0.0 %)	3 ms	850 ms
GET /actuator/health	319	0 (0.0 %)	110 ms	1300 ms
Consolidado Global	2446	0 (0.0 %)	5 ms	1100 ms



Figura 4: Reporte gráfico de Locust evidenciando el Throughput constante y 0 % de fallas bajo carga.

5.4. Tablero de Grafana con las 6 Vistas Operativas

Se diseñó y exportó el tablero oficial en `ops/grafana/pfc-dashboard.json`, cubriendo las **6 vistas obligatorias** (Figura 5):

1. **Tasa de Peticiones/seg (RPS):** Throughput por microservicio y balanceador HAProxy.
2. **Latencias Percentiles P50, P95 y P99:** Histogramas calculados vía PromQL sobre peticiones HTTP.
3. **Tasa de Errores 4xx y 5xx:** Monitoreo perimetral de rechazos de autenticación y fallas de backend.
4. **Disponibilidad y Consenso Raft:** Estado del líder etcd (`etcd_server_has_leader`), propuestas comprometidas y conexiones HikariCP.
5. **Uso de CPU y RAM por Contenedor:** Métricas extraídas en tiempo real por cAdvisor.
6. **Latencia Extremo a Extremo (E2E) Móvil:** Tiempos de respuesta en rutas de sincronización docente y HAProxy backend.



Figura 5: Tablero de Grafana (`pfc-dashboard.json`) operando en tiempo real durante la prueba de carga.

5.5. Evaluación de Calidad ISO/IEC 25010:2023 y Plan de Mejora

La Tabla 3 sintetiza la evaluación del sistema frente a los atributos de calidad de la norma ISO/IEC 25010:2023.

Tabla 3: Matriz de Evaluación de Calidad ISO/IEC 25010:2023 y Plan de Mejora Técnico.

Característica	Métrica	Valor Real	Objetivo	Estado	Plan de Mejora Técnico
1. Disponibilidad	Uptime en pruebas	100 %	$\geq 99,5\%$	✓ Cumple	Despliegue Multi-AZ con auto-escalado (HPA) en Kubernetes y réplicas primario-standby para PostgreSQL.
2. Rendimiento	Latencia P_{95}	285 ms	< 500 ms	✓ Cumple	Incorporación de capa caché distribuida en memoria con Redis para catálogos curriculares (< 15 ms).
3. Fiabilidad	Tasa de error 5xx	0.0 % (2446 req)	$< 1,0\%$	✓ Cumple	Patrón Circuit Breaker y Rate Limiting con Resilience4j en Gateway para mitigar fallas en cascada.
4. Mantenibilidad	Cobertura JaCoCo	83.0 %	$\geq 70,0\%$	✓ Cumple	Desacoplar generación de reportes PDF hacia workers asíncronos con colas RabbitMQ.
5. Seguridad	Rechazo sin JWT	100 % (HTTP 401)	100 %	✓ Cumple	Rotación dinámica de secretos vía HashiCorp Vault y listas de revocación de tokens en Redis.

6. Reflexión Ética sobre Tratamiento de Datos de Menores

El desarrollo y despliegue de sistemas de gestión escolar conlleva una responsabilidad ética ineludible, dado que los expedientes procesados contienen información de carácter personal y sensible correspondiente a niños, niñas y adolescentes matriculados en la institución. Bajo este marco, el equipo adoptó

como guía deontológica el **Código de Ética y Práctica Profesional de la Ingeniería de Software** promulgado conjuntamente por ACM e IEEE-CS (1999) [20].

En concordancia con el **Principio 1 (Interés Público)**, los ingenieros de software deben anteponer el bienestar de los usuarios y la sociedad a cualquier conveniencia técnica o cronograma de entrega. En el SGA, esto se materializó en la priorización absoluta de la resolución de la vulnerabilidad IDOR detectada en el módulo de soporte: un fallo de diseño previo permitía que usuarios autenticados iteraran identificadores para acceder a historiales técnicos donde constaban datos de menores. Subsanan esta brecha implementando un control de acceso de dos niveles no constituyó una simple optimización de código, sino el cumplimiento del deber ético de prevenir la divulgación no autorizada de información estudiantil.

Asimismo, en atención al **Principio 3 (Calidad del Producto)**, se aseguró que toda transmisión perimetral e interna estuviera cifrada y autenticada mediante tokens JWT firmados criptográficamente, eliminando credenciales en texto plano del código fuente [15]. Finalmente, el **Principio 6 (Profesión)** motivó la implementación de un registro de auditoría inmutable con relojes de Lamport en el módulo de secretaría, garantizando que ninguna modificación a calificaciones o registros de matrícula pueda ejecutarse de forma arbitraria o anónima, salvaguardando así la confianza institucional y la integridad del expediente educativo.

7. Amenazas a la Validez

Siguiendo las directrices metodológicas para la experimentación en ingeniería de software [21, 22], se identifican las siguientes amenazas a la validez del estudio experimental realizado:

7.1. Amenazas a la Validez Interna

1. **Efecto de Calentamiento de la JVM (*Warm-up Effect*):** La compilación *Just-In-Time* (JIT) de la máquina virtual Java influye en las mediciones iniciales de latencia. Para mitigar este sesgo, se ejecutaron fases de precalentamiento con peticiones previas antes de capturar la ventana de medición formal de 60 segundos.
2. **Contención y Recursos en Entorno de Host Local:** Las pruebas de carga se ejecutaron contra contenedores Docker en un mismo host físico, lo que puede inducir contención de CPU entre el generador de carga (Locust) y los microservicios. Se mitigó monitoreando con cAdvisor que el uso global de CPU del host no superara el 75 %.
3. **Brechas Iniciales de Instrumentación de Percentiles:** La falta previa de la propiedad `percentiles-histogram` en Micrometer generaba lecturas nulas en P95/P99. Se mitigó reescribiendo la configuración y verificando la aparición de las series temporales con `promtool` y Grafana.

7.2. Amenazas a la Validez Externa

1. **Patrones de Tráfico Sintético vs. Comportamiento Humano Real:** Los usuarios simulados en Locust siguen distribuciones de espera fijas y aleatorias uniformes, las cuales pueden diferir de los patrones estacionales reales de los períodos de matrícula.
2. **Topología de Red y Latencia WAN:** El despliegue de pruebas se realizó en una red local puente (*bridge*), por lo que las latencias de red (< 1 ms) no reflejan la variabilidad inherente a conexiones celulares móviles (3G/4G/5G) de los docentes en zonas rurales.

8. Conclusiones y Trabajo Futuro

8.1. Conclusiones Generales del Proyecto

La ejecución de la Entrega 4 permitió validar empíricamente que una arquitectura de microservicios distribuida, complementada con un API Gateway bien configurado y una capa de observabilidad

multi-fuente, es capaz de brindar alta disponibilidad, consistencia y seguridad para la gestión escolar. La integración de gRPC demostró reducir la latencia inter-servicio a niveles inferiores a 2 ms, mientras que el particionamiento multi-esquema en PostgreSQL combinó aislamiento de dominio con simplicidad operativa.

8.2. Conclusiones Individuales de los Integrantes

8.2.1. Conclusión Individual – Pedro Leonardo Castro López (Arquitecto y Líder de Proyecto)

El diseño, refactorización e implementación de los patrones de diseño GoF (Repository, Strategy, Template Method, Observer y Facade) en el módulo central `sga-principal` evidenció la relevancia de desacoplar las capas de negocio de la infraestructura en sistemas críticos. Implementar barreras de calidad automatizadas mediante un pipeline CI/CD en GitHub Actions con cuatro trabajos encadenados y certificar una cobertura del 88 % con JaCoCo bajo reglas estrictas (`<goal>check</goal>`) garantizó que ningún cambio se despliegue en producción sin superar pruebas exhaustivas. Asimismo, la instrumentación de métricas de negocio con Micrometer y Prometheus permitió contar con observabilidad en tiempo real sobre transacciones académicas y matrículas.

8.2.2. Conclusión Individual – Keyla Betzabe Bedon Viteri (Desarrolladora Backend y Móvil)

El desarrollo del microservicio docente sobre Python/Django y la aplicación móvil bajo el paradigma *offline-first* ratificó que los sistemas escolares distribuidos deben tolerar desconexiones sin comprometer la integridad. La implementación del conmutador de auditoría criptográfica con encadenamiento SHA-256 y marcas lógicas de Lamport garantizó que cualquier manipulación no autorizada en calificaciones sea detectable de forma inmediata. Asimismo, alcanzar una cobertura del 73 % con `pytest-cov` certificó la fiabilidad en la ejecución de servicios transaccionales y la comunicación de alto rendimiento vía gRPC.

8.2.3. Conclusión Individual – Ernesto Gregory Luna Mora (Responsable de Calidad y Gateway)

La implementación del API Gateway perimetral con HAProxy y la configuración de pruebas de integración con JaCoCo al 70 % en el microservicio de secretaría demostraron ser esenciales para gobernar el flujo de tráfico entrante y blindar la seguridad institucional. La evaluación formal bajo la norma ISO/IEC 25010:2023 con datos medidos en pruebas de carga y el rechazo estricto (HTTP 401) ante peticiones sin credenciales JWT válidas certificaron la robustez del sistema, asegurando la confidencialidad de los expedientes estudiantiles y la alta disponibilidad ante fallos.

8.2.4. Conclusión Individual – Juliana Romina Emanuel Pino (Documentalista y Observabilidad)

El desarrollo de esta entrega permitió comprender que la observabilidad y la seguridad no son complementos accesorios, sino pilares estructurales que deben verificarse de forma empírica. Durante la implementación de `microservicio-soporte`, la corrección de la vulnerabilidad IDOR y el saneamiento de secretos JWT demostraron que la protección de datos sensibles de menores exige controles de acceso a nivel de recurso y no sólo autenticación general. Asimismo, en la ampliación del tablero de Grafana con las 6 vistas operativas, se comprobó que las métricas promedio ocultan problemas graves de latencia que únicamente los percentiles de cola (P95/P99) y cAdvisor revelan con precisión bajo estrés concurrente.

Referencias

- [1] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. Sebastopol, CA, USA: O'Reilly Media, 2 ed., 2021.
- [2] G. Yu, P. Chen, and H. Zheng, "Microservice architecture in reality: An industrial survey," *IEEE Transactions on Software Engineering*, vol. 49, no. 4, pp. 1894–1910, 2021.
- [3] B. Burns, *Designing Distributed Systems: Patterns and Paradigms for Scalable, Reliable Services*. O'Reilly Media, 2 ed., 2025.
- [4] T.-H. Chen, S. Shang, M. Jiang, and A. E. Hassan, "An empirical study of log-based microservice observability: Insights from practitioners," *IEEE Transactions on Software Engineering*, vol. 48, no. 11, pp. 4633–4648, 2022.
- [5] L. Wu, J. Bogatinovski, S. Nedelkoski, J. Tordsson, and O. Kao, "Performance analysis and root cause localization for microservice systems using tracing and metrics," *IEEE Transactions on Network and Service Management*, vol. 20, no. 3, pp. 3120–3135, 2023.
- [6] S. Nedelkoski, J. Bogatinovski, A. Acker, J. Becker, J. Cardoso, and O. Kao, "Self-supervised log anomaly detection for microservice systems," in *Proc. 2021 IEEE International Conference on Web Services (ICWS)*, pp. 11–20, 2021.
- [7] Z. Li, J. Chen, R. Jiao, N. Wang, J. Chen, Z. Wang, H. Shu, and Y. Feng, "Holistic fine-grained anomaly detection and root cause analysis for microservice systems," in *Proc. 2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE)*, pp. 221–232, 2021.
- [8] P. Liu, H. Xu, Q. Ouyang, R. Jiao, Z. Chen, S. Zhang, and Y. Kang, "Unsupervised anomaly detection for microservices through multi-source observability data fusion," *IEEE Transactions on Services Computing*, vol. 17, no. 1, pp. 182–196, 2024.
- [9] C. Zhang, X. Peng, C. Sha, K. Zhang, Z. Fu, X. Wu, Q. Lin, and D. Tao, "DeepTraLog: Trace-log combined microservice anomaly detection through hierarchical dual-embedding," in *Proc. 2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE)*, pp. 623–634, 2022.
- [10] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *Proc. USENIX Annual Technical Conference (USENIX ATC)*, pp. 305–319, 2014.
- [11] J. Wang, Y. Zhang, Y. Ding, Z. Liu, and C. Wu, "Optimizing Raft consensus in high-performance distributed key-value stores," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 5, pp. 1450–1463, 2023.
- [12] S. Li, C. Guo, Y. Zhou, X. Niu, and C. Nie, "Chaos engineering in cloud-native microservice systems: A systematic review," *IEEE Software*, vol. 40, no. 2, pp. 64–72, 2023.
- [13] W. Zheng, Y. Li, Z. Fang, M. Li, and K. Zhang, "Automated resilience testing and circuit breaking verification in distributed microservices," *IEEE Transactions on Reliability*, vol. 73, no. 2, pp. 890–904, 2024.
- [14] P. Valderas, V. Torres, and E. Manso, "Security in microservice architectures: A systematic mapping study," *IEEE Access*, vol. 12, pp. 12845–12863, 2024.
- [15] M. Alharbi, A. Aljohani, and B. Alotaibi, "Zero-trust security architecture for cloud-native microservices: Token-based authentication and role access," *IEEE Internet of Things Journal*, vol. 12, no. 3, pp. 2410–2423, 2025.
- [16] M. B. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," Tech. Rep. RFC 7519, IETF, 2015.

- [17] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, Irvine, CA, USA, 2000.
- [18] N. Provos and D. Mazières, “A future-adaptable password scheme,” in *Proc. USENIX Annual Technical Conference (USENIX ATC)*, pp. 81–92, 1999.
- [19] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O’Reilly Media, 2017.
- [20] ACM/IEEE-CS Joint Task Force on Software Engineering Ethics and Professional Practices, “Software engineering code of ethics and professional practice,” *IEEE Computer*, vol. 32, no. 10, pp. 84–89, 1999. DOI: <https://doi.org/10.1109/MC.1999.796142>.
- [21] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Berlin, Germany: Springer, 2012.
- [22] P. Runeson and M. Höst, “Guidelines for conducting and reporting case study research in software engineering,” *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009.

Anexos: Preguntas Técnicas de Evaluación de la Arquitectura

A. ¿Cuál fue el cuello de botella identificado durante el proceso de pruebas?

El cuello de botella principal no fue de capacidad de cómputo en CPU, sino de **instrumentación y contención de conexiones**: inicialmente el microservicio no exportaba los *buckets* de histograma de Micrometer (resuelto con `percentiles-histogram.http.server.requests=true`), y el *pool* HikariCP operaba con el límite por defecto de 10 conexiones, incrementando la latencia en transacciones concurrentes dependientes de PostgreSQL.

B. ¿Cómo se comportó el servicio bajo una carga de 50 usuarios concurrentes durante 60 segundos?

El sistema procesó **2446 peticiones con un 0 % de tasa de error**. Los endpoints livianos (`/health`) mantuvieron medianas de 3 ms, mientras que las consultas a base de datos (`/api/soporte/tickets`) alcanzaron una mediana de 130 ms y un percentil P99 de 1300 ms, evidenciando estabilidad sin pérdida de transacciones.

C. ¿Qué *quality gates* se identifican o recomiendan a partir de este ejercicio?

Se definieron tres controles indispensables: (1) **Umbral de cobertura JaCoCo $\geq 70\%$** y paso de tests unitarios bloqueante en GitHub Actions; (2) **Gate de latencia P99 y tasa de error $< 1\%$** en pruebas de carga automatizadas; y (3) **Escaneo automatizado de secretos criptográficos** (*secret scanning*) para impedir commits con claves JWT en código fuente.

D. ¿Qué mejora concreta se propone a partir de los hallazgos de esta entrega?

Se propone (1) dimensionar explícitamente el *pool* HikariCP y desplegar una capa de caché distribuida en memoria con **Redis** para catálogos curriculares y validación rápida de tokens; y (2) implementar *Distributed Tracing* con **OpenTelemetry / Jaeger** para correlacionar trazas entre el cliente móvil, HAProxy y los microservicios backend.